

Self-Level Inheritance Verification — Decomposing B1.05 *Self as Integrated Whole* by Formalizing How Series A's Operational Tests A5.01–A5.16 Are Applied at Self Level in Paper 2 Deployments to Verify That Paper 1 Commitments Hold Operationally for Selves per B1.20 Recursive Inheritance, Paralleling Cell-Level Verification per B2.14 and Aspect-Level Verification per B2.19, Closing the B1.05 Decomposition

Author: Wenxin Li (Independent Researcher) **ORCID:** 0009-0004-8065-3235 **Date:** May 8, 2026
License: Creative Commons Attribution 4.0 International (CC BY 4.0)

* * *

Attribution

This work derives from and formalizes the instinct/reasoning separation pattern introduced in *The Instinct/Reasoning Separation Outside the Model* (Li, April 2026), the second paper in the CKS theory series following *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize one operational variant within Paper 2's three-level structural commitment — **Self-level inheritance verification**, the application of Series A's operational tests A5.01–A5.16 at Self scope to verify that Paper 1's architectural commitments hold operationally for Selves per the recursive-inheritance commitment formalized in B1.20. This note closes the five-note decomposition of B1.05 (*Self as integrated whole*) and closes the level-decomposition sequence B2.07–B2.24.

Abstract

Paper 2 commits Paper 1's architectural properties to hold recursively at every structural level (cell, aspect, Self) — the recursive-inheritance commitment formalized in B1.20. The commitment is testable, but only if the architecture specifies what testing it operationally means at Self level. This note formalizes that operational meaning: the application at Self scope of the same Series A operational test suite (A5.01–A5.16) that B2.14 applies at cell scope and B2.19 applies at aspect scope. The note states the verification precisely, distinguishes it from single-level testing in conventional AI architectures, names the cognitive analog as conceptual scaffold while preserving the architectural substance, enumerates the inherited Paper 1 commitments the verification is responsible to, articulates operational implications including the lifecycle and evolution gates the verification supports, and provides an operational test for whether a given deployment instantiates the verification at Self scope. With this note, the B1.05 decomposition closes, and Phase B2 advances to the B1.06 *two layers within every cell* decomposition.

1. Why Self-level inheritance verification needs to be formalized as standalone operational variant

Paper 2 introduces three structural levels — cell, aspect, Self — and commits Paper 1's architectural properties to hold recursively at each (B1.20). The recursive commitment is what makes Paper 2 a coherent extension rather than a new architecture: the same defended properties continue to hold as the structure scales upward.

Recursive inheritance is, however, only operationally consequential if testable. A deployment that claims to inherit Paper 1's commitments at Self level but cannot demonstrate them operationally has made an architectural assertion that the deployment does not in fact instantiate. Demonstration requires operational tests; tests at the highest structural level require artifacts under test that exist at that level; and the test specifications must be consistent with those run at lower levels for the recursion to be a single coherent property rather than three independent ones.

Series A's operational tests A5.01–A5.16 are the canonical operational tests for Paper 1's commitments. Cell-level inheritance verification (B2.14) applies these tests at cell scope; aspect-level inheritance verification (B2.19) applies them at aspect scope. The architectural pattern is incomplete without the parallel application at Self scope; B1.20's recursive inheritance is only fully demonstrable when the same suite runs at all three levels. This note formalizes that final application, closing the B1.05 decomposition (B2.20 integrated whole, B2.21 integration architecture, B2.22 content-domain operationalization, B2.23 instinct/reasoning configuration, B2.24 inheritance verification) and closing the broader level-decomposition sequence B2.07–B2.24 covering cell, aspect, and Self decompositions in turn.

2. The architectural verification precisely stated

In a Paper 2 deployment, **Self-level inheritance verification** is the application of Series A's operational tests A5.01 through A5.16 at Self scope, with **Self-distinct semantics** — same test specifications, with the architectural artifacts under test being the Self-level artifacts named by B1.05, B2.21, B2.22, and B2.23.

The sixteen tests, applied at Self level, run as follows.

A5.01 — Inspect Right Test (per A2.01). A human with appropriate access can inspect the Self's substrate content within authorized scope: the Self integration architecture (per B2.21), the Self-level instinct/reasoning configuration (per B2.23), Self-level rules, Self-level outputs, and the aspect-membership relationships that constitute the Self.

A5.02 — Modify Right Test (per A2.02). A human with appropriate access can modify Self-level rules within authorized scope, with the change taking effect as Self-level substrate state.

A5.03 — Override Right Test (per A2.03). A human with appropriate access can override specific Self-level decisions and outputs without justification to the architecture, even where the decision derived from cross-aspect coordination.

A5.04 — Rule Authoring Test (per A2.04). Self behavior is governed by authored rules: the Self integration rules, the Self-level orchestration rules, and the instinct/reasoning configuration are all authored under human authority before they take effect.

A5.05 — Mediator Role Test (per A2.18–A2.23, Properties A–E). LLM consultation at Self level — for example, instinct-layer consultations whose outputs cross aspect boundaries — respects the five mediator properties: bounded scope, deterministic substrate effects, non-authoritative without rule-authorized commit, traceable consultation, and substitutable model.

A5.06 — Cell-Behavior-Determinism Test, adapted to Self-Behavior-Determinism (per A1.10). Self behavior given inputs is deterministic at the coordination layer: the same Self state and the same inputs produce the same Self-level substrate effects, allowing for the bounded non-determinism the determinism contract permits.

A5.07 — Read-Determinism Test (per A1.10). Reads of Self-level substrate content return consistent results across consultations, free of LLM-mediated re-derivation drift.

A5.08 — Provenance-Completeness Test (per A1.07, A2.40). Self-level operations carry the six-field provenance record: who/what/when/why/under-what-rule/with-what-result.

A5.09 — Four-Accountability-Questions Test (per A1.07). Every Self-level operation answers who acted, what was acted on, when it occurred, and why under which authoring authority.

A5.10 — Source-of-Truth-Five-Categories Test (per A1.08). Self-level content fits the five categories: structured entities, relationships, decisions, rationale, and conflicts; nothing about Self-level state lives only in agent memory or only in LLM weights.

A5.11 — Tool-Agnosticism-Migration Test (per A1.05). The Self continues to operate correctly under substrate-technology migration. This is particularly significant at Self level because LLM-vendor migration is a Self-level configuration change governed by the instinct/reasoning configuration (per B2.23): the test verifies that the Self's behavior at the coordination layer is preserved across instinct-layer substitutions performed under governance.

A5.12 — Linear-Cost-Scaling Test (per A1.06). Self-level governance cost — exercised at the two governance moments named in A1.01 — does not grow with Self size, only with rule variety and intervention frequency.

A5.13 — Conflict-Coexistence Test (per A1.03). The Self handles conflicts in accordance with the conflict-as-first-class commitment, with particular emphasis at Self level on **cross-aspect conflicts**: when two aspects within the Self produce conflicting outputs or hold conflicting state, the Self's integration architecture (per B2.21) registers and surfaces the conflict rather than auto-resolving it, leaving resolution to human authority or to authored rules under human authority.

A5.14 — Composition-Requirements-Five Test (per A1.13). Self-level composition — the composition of aspects into a Self — meets the five composition requirements: addressability, governed access, conflict-aware composition, retraceable composition, and tool-agnostic composition.

A5.15 — Pattern-Mapping Test (per A2.92–A2.94). Self-aspect relationships fit one of the named patterns (Pattern A, Pattern B, Pattern C) for the deployment's chosen composition mode, with the chosen pattern itself recorded as substrate content.

A5.16 — Reproducibility Test. Self operations replay deterministically over recorded inputs and Self state at the coordination layer, recovering the same coordination-layer outputs they originally produced.

The verification suite is itself **governed per A1.01**: which tests run, on what schedule, with what parameter values, under what authority — these are themselves substrate content authored under governance. Verification events are **recorded per A2.40**: the same six-field provenance applies. The architectural feature making the recursion across cell, aspect, and Self levels operationally meaningful is the **consistency of the test specifications across levels**: the suite is the same suite, applied with Self-distinct semantics rather than with Self-distinct tests.

3. What makes Self-level inheritance verification architecturally distinctive

Conventional AI architectures often verify component-level properties (per-model evaluation, per-component unit testing) or system-level properties (end-to-end integration testing) without a recursive standard test suite at multiple levels. The architectural commitment underwriting Self-level inheritance verification is different in structure: the same standard test suite runs at every level of the architecture's structural commitment, and that recursion is what makes B1.20 fully demonstrable.

Self-level verification specifically covers architectural artifacts that exist only at Self scope: the integration architecture (B2.21), the instinct/reasoning configuration as the Self holds it (B2.23), and the multiple-aspect coexistence that defines the Self (B1.05). Cell-level verification cannot reach these; aspect-level verification covers single-aspect content but not the coordination across aspects that the Self introduces. Self-level verification is therefore not redundant with the lower levels — it covers content the lower levels cannot reach, while applying the same test specifications.

The standard test specifications are adapted to Self-distinct semantics in the sense that A5.01's "substrate content" at Self level means Self-level substrate content (integration architecture, Self-level rules, aspect membership), not cell-level substrate content; A5.13's conflicts at Self level mean cross-aspect conflicts specifically; A5.11's migration at Self level includes LLM-vendor migration as a Self-scope configuration change. The tests are the same; their semantic instantiation is Self-distinct.

4. The cognitive analog

Self-level inheritance verification has a familiar analog in modular software architecture: integrated-system testing applied at the integrated level using specifications consistent with those used at component level. The analog functions as conceptual scaffold and is useful for that purpose. It is, however, only a scaffold. The architectural substance is the recursive application of Series A's specific test suite — the sixteen tests deriving from Paper 1's specific commitments — at Self scope.

5. The inherited Paper 1 commitments

Self-level inheritance verification is responsible to and inherits from a specific subset of Paper 1's defended commitments, named through their A-series notes.

- **A1.01 — human-governed.** Verification is itself governed: test specifications, parameters, and run schedule are substrate content under human authority.
- **A1.07 — path retraceability.** Verification events are recorded with the six-field provenance vocabulary.
- **A1.10 — determinism contract.** Verification tests are themselves deterministic at the coordination layer.
- **A1.13 — composition requirements.** Verification covers the five composition properties at Self scope, where aspects compose into a Self.
- **A2.04 — rule authoring.** The verification rules themselves are authored.
- **A2.40 — provenance.** Verification operations have the same six-field record as any other Self-level operation.
- **A5.01–A5.16 — operational tests.** The test specifications themselves are inherited references; this note does not redefine them, only applies them at Self scope.
- **B1.20 — recursive inheritance.** The recursive commitment is what verification establishes operationally; without it, Self-level verification would be ad hoc rather than the operational counterpart of an architectural commitment.
- **A1.03 — conflict-as-first-class.** Verified at Self level specifically through A5.13, with cross-aspect conflicts as the Self-distinct semantic content.

6. Operational implications

Naming Self-level inheritance verification as a standalone operational variant has six consequences for deployments.

Verification is configured per Self type. Different Self configurations may have different test parameter values for the same standard tests. Specifications are constant; parameters are deployment configuration.

Verification gates Self birth (per B1.09). A newly instantiated Self must pass Self-level verification before entering operational use.

Verification gates Self modification. When a Self is modified — integration architecture changed, instinct/reasoning configuration adjusted, aspects added or removed — the modified Self is re-verified.

Verification gates Self-level vertical evolution (per B1.16). When vertical evolution restructures the Self, the restructured Self is re-verified.

Verification supports Self lifecycle decisions (per B1.11). Selves that fail verification may be subject to functional-obsolescence death: a Self that cannot demonstrate its inherited Paper 1 commitments is, by the architecture's own discipline, no longer fit to operate.

Verification interacts with mating (per B1.10). Selves created through any of the three mating patterns must pass Self-level verification as part of birth. Cross-partner verification follows the authority distribution formalized in A2.47.

Verification operates concurrently with cell-level (B2.14) and aspect-level (B2.19) verification. All three levels are required for full inheritance assurance. Self-level verification covers content the lower levels cannot reach but does not cover content the lower levels do reach. Together the three cover the level scopes per B2.07.

Verification failures, at any level, trigger governance actions per A2.03 (override) or A2.04 (rule authoring): a human exercises override authority over a failing Self-level decision, or authors new rules that re-shape the Self's behavior such that subsequent verification passes. The verification record itself is substrate content and is itself inspectable, modifiable, and overridable per A1.01.

7. Limits

The operational variant has bounded scope. Naming the limits is what keeps the standalone framing from drifting into something stronger than the source paper supports.

Self-level verification does not replace cell-level or aspect-level verification. Distinct levels have distinct verification scopes. A deployment that runs Self-level verification but skips cell-level or aspect-level verification has not satisfied B1.20's recursive commitment. The recursion is the architecture; partial recursion is a different architecture.

Self-level verification does not eliminate domain-specific Self testing. The sixteen tests verify inherited architectural properties. A Self deployed for a particular domain may also require domain-specific operational testing — additional, not in tension with inheritance verification.

Self-level verification does not prescribe specific test parameters. Parameter values for the standard tests are deployment configuration authored under human authority. The standard fixes the test specifications, not the parameter values.

Self-level verification does not guarantee behavioral correctness. It verifies architectural properties, not domain-output correctness.

Self-level verification is not a single test but the application of A5.01–A5.16 at Self scope. A deployment that runs only a subset of the suite has not satisfied the verification.

Self-level verification does not eliminate verification gates for instinct (per B2.06). The instinct layer has its own verification structure for LLM behavior. Both run; neither substitutes for the other.

Self-level verification is intra-Self. Inter-Self verification — how multiple Selves are jointly verified, how regulatory bodies verify across Selves — is out of scope for Paper 2 and therefore for this note.

8. Operational test

A deployment instantiates Self-level inheritance verification if and only if all of the following are true at all times during the Self's operational existence:

1. Tests A5.01 through A5.16 are applied at Self scope, with the architectural artifacts under test being the Self's substrate content (integration architecture per B2.21, instinct/reasoning configuration per B2.23, Self-level rules, Self-level outputs, aspect-membership relationships).
2. The test specifications applied at Self level are the same specifications as those applied at cell level per B2.14 and at aspect level per B2.19; only the architectural artifacts under test differ.
3. A5.13 at Self level specifically verifies cross-aspect conflict-coexistence, with the Self's integration architecture registering and surfacing rather than auto-resolving conflicts among aspects.
4. The test specifications, parameter values, and run schedule are themselves substrate content under human authority per A1.01.
5. Verification events are recorded with the A2.40 six-field provenance vocabulary.
6. Self birth, Self modification, and Self-level vertical evolution gate on successful verification before operational use.
7. Verification failures trigger A2.03 override or A2.04 rule-authoring actions, with the failure record and the response itself recorded as substrate content.
8. Self-level verification runs concurrently with cell-level (per B2.14) and aspect-level (per B2.19) verification, not in place of them.

A deployment that fails any of (1)–(8) does not instantiate Self-level inheritance verification, even if it runs verification at one or both other levels. The architectural commitment is to the recursive application of the standard suite at all three levels.

9. Conclusion

Naming Self-level inheritance verification as a standalone operational variant gives downstream implementers a precise specification of what their Self-level verification mechanism must satisfy and how it must relate to the parallel mechanisms at cell scope (B2.14) and aspect scope (B2.19). The substance of the variant is the recursive application of Series A's sixteen operational tests at Self scope, with Self-distinct semantics in which the architectural artifacts under test are the Self-level artifacts named by B1.05, B2.21, B2.22, and B2.23.

The variant closes the B1.05 *Self as integrated whole* decomposition (B2.20 integrated whole, B2.21 integration architecture, B2.22 content-domain operationalization, B2.23 instinct/reasoning configuration, B2.24 inheritance verification) and closes the broader level-decomposition sequence B2.07–B2.24 covering all three structural levels. With this note, Phase B2 advances from the level-decomposition sequence to the ****B1.06 *two layers within every cell***** decomposition with subsequent notes B2.25–B2.29.

Subsequent work that implements, extends, or argues against the Self-level verification commitment should use "Self-level inheritance verification" in the sense formalized here. Subsequent work that uses the term differently is using a different concept, and the difference should be named.

* * *

Source paper

Li, W. (2026). *The Instinct/Reasoning Separation Outside the Model: Extending the Coordination Knowledge Substrate Pattern to AI Selves under Human Governance*. April 2026. ORCID: 0009-0004-8065-3235.

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems*. April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Self-Level Inheritance Verification — Decomposing B1.05 Self as Integrated Whole by Formalizing How Series A's Operational Tests A5.01–A5.16 Are Applied at Self Level in Paper 2 Deployments to Verify That Paper 1 Commitments Hold Operationally for Selves per B1.20 Recursive Inheritance, Paralleling Cell-Level Verification per B2.14 and Aspect-Level Verification per B2.19, Closing the B1.05 Decomposition*. May 8, 2026. ORCID: 0009-0004-8065-3235.